

nyaml: Format Converter for the NeXus Data Model.

Rubel Mozumder ¹, Lukas Pielsticker ^{1,3}, Markus Kühbach ¹, Andrea Albino ¹, Florian Dobener ¹, Sherjeel Shabih ¹, Christoph Koch ¹, Heiko B. Weber ², José Antonio Márquez Prieto ¹, Claudia Draxl ¹, and Sandor Brockhauser ¹

¹ Physics Department and CSMB, Humboldt-Universität zu Berlin, Berlin, Germany ² Lehrstuhl für Angewandte Physik, Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany ³ Department Heterogeneous Reactions, Max Planck Institute for Chemical Energy Conversion, Mülheim an der Ruhr, Germany

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a

Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

The NeXus scientific data format ([Könnecke et al., 2015](#)) is a widely adopted standard for organizing and sharing scientific data, particularly in the field of materials characterization. NeXus definitions specify the hierarchical structure and semantics of valid NeXus files and are written in XML ([Bray et al., 2008](#)) using the NeXus Definition Language (NXDL), which itself is specified using XSD (XML Schema Definition).

nyaml is a Python-based tool with both a command-line interface and an importable API that facilitates the conversion between NXDL XML and a simplified YAML ([Ben-Kiki et al., 2009](#)) representation. YAML's indentation-based syntax enhances human readability and simplifies manual editing. By providing a reliable, lossless round-trip conversion between XML and YAML, **nyaml** enables developers to edit NeXus definitions more efficiently without sacrificing structural or semantic fidelity.

Statement of need

The NeXus data format standard, which was originally introduced for neutron, X-ray, and muon science, has in recent years seen a significant enhancement across diverse scientific domains. The growth of both the standard and the number of NeXus definition developers makes it all the more important to ensure that the development process is both user-friendly and resilient to errors. The existing representation of NeXus definitions in XML offers structural rigor through the NeXus Definition Language (NXDL) and a well-defined hierarchy for metadata and data types. However, it is verbose and can be difficult to edit by hand. Writing and maintaining NXDL files often involves dealing with deeply nested elements and strict syntax, which can be error-prone and time-consuming, especially for users who are not familiar with XML development. **nyaml** addresses these challenges by enabling the development of NeXus definitions in a cleaner, YAML-based format while preserving the full structure, semantics, and developer comments of the original XML.

nyaml Converter

The **nyaml** tool is a Python package, containing the several modules, that provides a command line interface for converting NeXus application definitions or base classes from YAML format (file with an **.yaml** extension) into the XML format (file with an **.nxdl.xml** extension) and vice-versa. To write a NeXus application definition or base class in YAML format, the **nyaml**

40 package introduces a set of the keywords and syntactic rules (see [Documentation](#)) that are
41 specific to the NXDL (NeXus Definition Language) in the YAML format.

42 The **nyaml** package is designed to be used as a command line tool, but it can also be utilised
43 as a Python module for programmatic use. The package is published in PyPI and therefore
44 can be installed using python package manager. The converter command is invoked by the
45 **nyaml2nxdl** registered in the **nyaml.cli** module ([team, 2025](#)). In that module, the function
46 **launch_tool** decides upon the input file type which what conversion (either from YAML to
47 XML or XML to YAML) will be invoked and execute the corresponding pipeline of the workflow
48 ([Figure 1](#)).

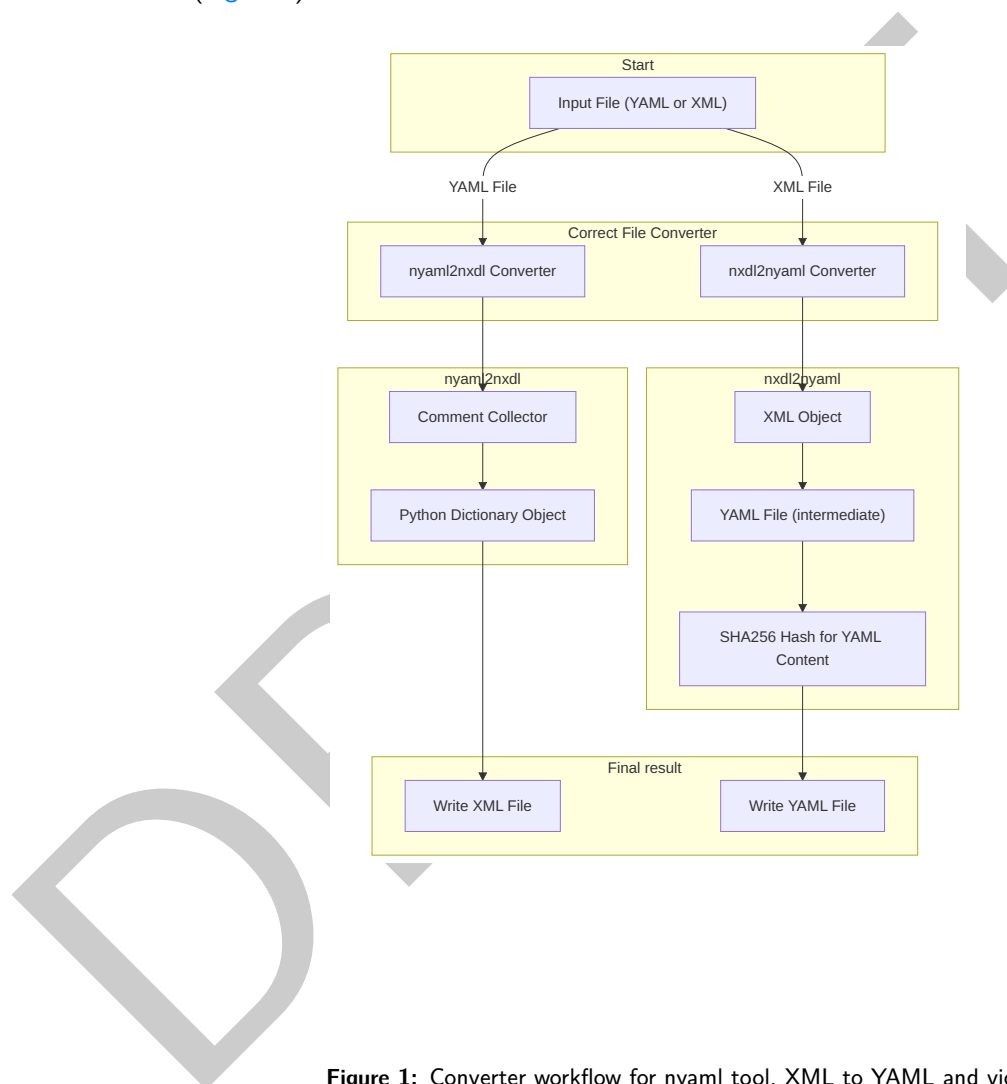


Figure 1: Converter workflow for nyaml tool, XML to YAML and vice-versa

49 YAML to XML

50 The **nyaml.nyaml2nxdl** contains the necessary functions, classes, and methods to implement
51 the algorithm for converting the NXDL schema from YAML to XML. The function **nyaml2nxdl**
52 is the main entry point for executing the conversion from YAML to XML with the execution in
53 three sequential tasks:

- 54 1. using **PyYAML** ([YAML & communities, 2024](#)) collects and tracks comments in YAML
55 file,
- 56 2. parses the YAML file into a Python dictionary object, and finally,

57 3. writes the XML tree in a file in accordance with the NXDL concepts including the
58 comments from the first step.

59 The conversion algorithm interprets the keywords and syntactic rules to read the NXDL in the
60 YAML format to transcode from the YAML to the XML representation. Taking leverage of
61 the NeXus NXDL rules, the conversion detect the possible inconveniences in the YAML file
62 and raises an error or warning if the NXDL rules are not properly followed in YAML.

63 XML to YAML

64 The `nyaml.nxdl2yaml` module contains the necessary functions, classes, and methods to
65 implement the algorithm for converting the NXDL schema from XML format to YAML format.
66 The class `Nxdl2yaml` is the main entry point containing methods and global variables for
67 implementing the conversion algorithm from XML to YAML with the execution of three tasks:
68 1. using `lxml (?)` parses the XML file into an XML tree object, 2. writes the YAML file from
69 the XML tree in accordance with the YAML specific keywords and rules, and 3. creates a hash
70 from the generated YAML content from task 2 using the SHA256 algorithm and extends the
71 final YAML file concatenating the hash and original xml content as comment respectively at
72 the end.

73 Attaching the XML content with a hash at the end of the YAML file allows for a lossless
74 round-trip conversion if no modifications are made in the YAML content (as SHA256 creates
75 the same hash from the same YAML content). Such caching mechanism offers clean and
76 efficient conversion XML -> YAML -> XML and thus cleaner comparison of the XML content
77 in version control systems like Git.

78 Evaluation from NAIC

79 The NeXus International Advisory Committee (NIAC) is the governing body responsible for over-
80 seeing the development and maintenance of the NeXus data standard. A core responsibility of
81 the NIAC is the stewardship of the NeXus Definition Language (NXDL), the XML-based schema
82 that defines the hierarchical structure and semantics of NeXus data files [Koennecke:2015].
83 As part of its mission to facilitate the standardization of NeXus definitions in NXDL, NIAC
84 has recently reviewed and formally accepted `nyaml`. Following a successful evaluation, NIAC
85 has approved `nyaml` and endorsed it as the recommended tool for preparing NeXus definition
86 proposals. In support of this decision, the official NeXus definition repository was updated to
87 integrate `nyaml` into its workflow through the addition of two makefile targets: 'make `nyaml`',
88 which converts existing definitions from the canonical `nxdl.xml` format into `.nyaml`, and 'make
89 `nxdl`', which detects modified or newly added `.nyaml` files and converts them back into valid
90 `nxdl.xml` format for submission and version control. This integration ensures that contributions
91 made in `.nyaml` are compatible with the existing XML-based infrastructure. The adoption
92 of `nyaml` by NIAC reflects an ongoing commitment to fostering community engagement and
93 modernising the technical tools underpinning the NeXus standard (Committee, 2025).

94 Acknowledgements

95 The `nyaml` software development is funded by the German National Research Data Infrastructure
96 (NDFI) consortia FAIRmat (Deutsche Forschungsgemeinschaft DFG, 460197019).

97 References

98 Ben-Kiki, O., Evans, C., & Ingerson, B. (2009). *YAML ain't markup language (YAML) (tm)*
99 *version 1.2*. YAML.org. <http://www.yaml.org/spec/1.2/spec.html>

- 100 Bray, T., Paoli, J., Sperberg-McQueen, C. M., Maler, E., & Yergeau, F. (2008). *Ex-*
101 *tensible markup language (XML) 1.0 (fifth edition)*. [https://www.w3.org/TR/2008/](https://www.w3.org/TR/2008/REC-xml-20081126/)
102 [REC-xml-20081126/](https://www.w3.org/TR/2008/REC-xml-20081126/).
- 103 Committee, N. I. A. (2025). *NIAC and NeXus documentation*. <https://www.nexusformat.org>
- 104 Könnecke, M., Akeroyd, F. A., Bernstein, H. J., Brewster, A. S., Campbell, S. I., Clausen, B.,
105 Cottrell, S., Hoffmann, J. U., Jemian, P. R., Männicke, D., Osborn, R., Peterson, P. F.,
106 Richter, T., Suzuki, J., Watts, B., Wintersberger, E., & Wuttke, J. (2015). The NeXus
107 data format. *Journal of Applied Crystallography*, 48(1), 301–305. [https://doi.org/10.](https://doi.org/10.1107/S1600576714027575)
108 [1107/S1600576714027575](https://doi.org/10.1107/S1600576714027575)
- 109 team, T. click development. (2025). *Command line interface creation kit* (latest). [https:](https://github.com/pallets/click)
110 [//github.com/pallets/click](https://github.com/pallets/click)
- 111 YAML, & communities, P. (2024). *PyYAML* (latest). <https://github.com/yaml/pyyaml>

DRAFT